

2.5.1 Contexto del sistema

“The context of the software provides a connection between the intended software and its external environment. This is crucial for understanding the context of the software in its operational environment and identifying its interfaces with the environment” [1].

Explicación

El contexto del sistema describe el entorno donde el software va a operar y marca los límites de lo que entra o no dentro de su alcance. Tener claro ese límite es necesario para entender cómo el software se relaciona con los elementos que lo rodean, como los usuarios, el hardware o sistemas externos con los que interactúa. Cuando ese contexto se define bien desde el inicio, el equipo puede identificar a todos los actores involucrados y sus necesidades reales, lo que ayuda a evitar que se desarrollen funcionalidades que nunca formaron parte del proyecto.

Ejemplo vinculado al PFC

En la red social para mascotas, el sistema opera dentro de un entorno donde conviven distintos tipos de usuarios: los dueños que buscan pareja para sus mascotas, las personas interesadas en adoptar y los administradores que gestionan la plataforma. Todo esto funciona a través de la web, lo que significa que debe andar bien tanto en el celular como en la computadora. Además, el sistema no trabaja solo, sino que se apoya en servicios externos de correo y mensajería para avisar a los usuarios cuando hay una coincidencia o alguien les envía una solicitud de contacto.

2.5.2 Actores del sistema (Stakeholders)

“Stakeholder is an individual or organization having a right, share, claim or interest in a system or in its possession of characteristics that meet their needs and expectations” [2].

Explicación

Los actores del sistema son todas las personas u organizaciones que de alguna forma se ven afectadas por cómo funciona el software o tienen algún interés en él. Cada uno llega con sus propias necesidades y expectativas, y eso hay que tomarlo en cuenta durante el desarrollo porque si no se identifican bien desde el principio, es fácil terminar construyendo algo que no le sirve a nadie. Conocer a los actores también ayuda a decidir qué funcionalidades son prioritarias y cuáles pueden esperar, lo que en la práctica hace que el producto final tenga más posibilidades de cumplir con lo que realmente se necesita.

Ejemplo vinculado al PFC

En esta red social los actores principales son bastante variados. Por un lado están los dueños de mascotas que usan la plataforma para encontrar una pareja para cruce, y por otro los adoptantes que buscan una mascota para llevarse a casa. También están los administradores, que son los que mantienen todo en orden y se aseguran de que se cumplan las reglas de la plataforma. Y luego hay un grupo adicional que podría sacarle bastante provecho al sistema, que son los veterinarios y criadores, quienes podrían usarlo como un canal más para ofrecer asesoramiento a los usuarios.

2.5.3 Límites del sistema (System Boundary)

“System boundary defines the limits that separate the system-of-interest from its external environment, establishing what is inside the system and what is outside” [2].

Explicación

Los límites del sistema definen hasta dónde llega la responsabilidad del software y dónde empieza la del usuario o de otros sistemas externos. Todo lo que cae dentro de esa frontera tiene que ser desarrollado, mientras que lo que queda afuera lo resuelve alguien más, ya sea un actor o un sistema de terceros. Tener eso claro desde el

inicio evita confusiones sobre quién hace qué y, sobre todo, previene que el proyecto empiece a crecer sin control porque nadie dejó establecido qué estaba incluido y qué no.

Ejemplo vinculado al PFC

Dentro del sistema entran cosas como el registro de perfiles de usuarios y mascotas, la publicación de anuncios de cruce y adopción, la mensajería interna entre usuarios y los filtros de búsqueda por raza, edad, ubicación y tamaño. Lo que ya queda fuera es todo lo que pasa en el mundo físico o legal: el traslado de las mascotas, revisar presencialmente su estado de salud, manejar los contratos de adopción o garantizar que el cruce o la adopción salgan bien. Eso no es responsabilidad del sistema sino de las personas involucradas.

2.5.4 Restricciones del sistema (Constraints)

“Constraint is an externally imposed limitation on the system, its design, or implementation or on the process used to develop or modify a system” [2].

Explicación

Las restricciones son condiciones que vienen de afuera y que el equipo de desarrollo no puede ignorar porque limitan directamente las decisiones de diseño e implementación. Pueden ser de tipo técnico, como tener que trabajar con una plataforma o tecnología que ya está definida, o más organizacionales, como el presupuesto que hay disponible o las fechas que no se pueden mover. Conocerlas y tenerlas documentadas desde el principio es clave porque le permite al equipo tomar decisiones que sean realistas y no terminar planificando algo que después no se puede ejecutar con los recursos que realmente hay.

Ejemplo vinculado al PFC

El sistema tiene que desarrollarse como una aplicación web responsive para que funcione bien tanto en celular como en computadora, eso no es opcional. También hay que tener en cuenta que al manejar datos personales de los usuarios, el sistema debe cumplir con la normativa de protección de datos, ya sea el GDPR o la ley local que aplique. A eso se le suma que el proyecto tiene cuatro meses para desarrollarse y un presupuesto ajustado, lo que en la práctica significa que no hay mucho margen para usar tecnologías de pago o contratar servicios externos costosos.

2.6 Calidad de requisitos e ISO/IEC 25010:2023

2.6.1 Calidad de requisitos

“A requirement is a statement which translates or expresses a need and its associated constraints and conditions. Requirements exist at different levels in the system structure” [2].

Explicación

La calidad de los requisitos tiene que ver con qué tan bien escritos y definidos están, es decir, que sean claros, completos, que no se contradigan entre sí y que se puedan verificar en algún punto del desarrollo. Un requisito bien redactado hace que todos los que participan en el proyecto entiendan exactamente lo mismo, sin que cada uno lo interprete a su manera. Esto importa bastante porque cuando los requisitos son ambiguos o incompletos, los errores aparecen más adelante y en ese punto ya son mucho más costosos de corregir que si se hubieran detectado desde el inicio.

Ejemplo vinculado al PFC

Un buen ejemplo de requisito de calidad en esta plataforma sería decir que el sistema debe permitirle a un usuario registrado filtrar las mascotas disponibles para adopción por ubicación dentro de un radio de 10 km, por especie entre perro y gato, y por tamaño entre pequeño, mediano y grande. Eso es muy diferente a escribir algo como "el sistema debe buscar mascotas fácilmente", que suena bien pero no le dice nada concreto al equipo

de desarrollo ni permite verificar después si se cumplió o no. Mientras más específico es el requisito, más fácil es saber si el sistema realmente lo está cumpliendo.

2.6.2 Functional Suitability (ISO/IEC 25010)

“Functional suitability is the capability of a product to provide functions that meet stated and implied needs of intended users when it is used under specified conditions” [3].

Explicación

La idoneidad funcional básicamente evalúa si lo que el sistema hace realmente le sirve al usuario, tanto en lo que pidió explícitamente como en lo que se da por sentado que debería funcionar. Esta característica se divide en tres partes: que el sistema tenga todas las funciones que se necesitan, que esas funciones den resultados correctos y que además estén pensadas para facilitar el trabajo del usuario y no complicárselo. Cuando un sistema cumple bien con esto, el usuario siente que el software hace exactamente lo que esperaba, ni más ni menos.

Ejemplo vinculado al PFC

En esta plataforma la idoneidad funcional se puede ver claramente cuando un dueño puede registrar a su mascota con toda su información, como la especie, raza, edad y fotos, publicar un anuncio indicando si busca cruce o quiere darla en adopción, recibir mensajes de otros usuarios que estén interesados y finalmente contactarlos para coordinar un encuentro. Esas cuatro cosas forman el flujo básico que el sistema tiene que garantizar. Si alguna falla o directamente no está, el sistema no está cumpliendo con lo que el usuario necesita y eso ya es un problema de idoneidad funcional.

2.6.3 Interaction Capability / Usability (ISO/IEC 25010)

“Interaction capability is the capability of a product to be interacted with by specified users to exchange information between a user and a system via the user interface to complete the intended task” [3].

Explicación

Básicamente la capacidad de interacción responde a una pregunta simple: ¿qué tan fácil es usar el sistema? La norma del 2023 actualizó este término y dejó atrás lo que antes llamaban usabilidad. Ahora engloba cosas como que el usuario entienda solo para qué sirve cada cosa, que no tarde días en aprender a manejarlo y que si va a cometer un error, el sistema lo frene antes de que pase algo grave. Al final cuando eso está bien resuelto, la gente usa el sistema sin frustrarse y sin necesitar que alguien le explique cómo funciona.

Ejemplo

La plataforma tiene que ser lo suficientemente intuitiva como para que una persona sin mucha experiencia con la tecnología pueda publicar el perfil de su mascota en menos de cinco minutos y sin tener que leer nada antes. El proceso de registro debe estar guiado paso a paso y apoyado con indicaciones visuales que hagan obvio qué hacer en cada momento. Y si el usuario intenta mandar un mensaje sin haber seleccionado una mascota primero, el sistema no puede simplemente no hacer nada, sino que tiene que avisarle con un mensaje claro y decirle exactamente qué paso se saltó para que pueda corregirlo sin frustrarse.

2.6.4 Performance Efficiency (ISO/IEC 25010)

“Performance efficiency is the capability of a product to perform its functions within specified time and throughput parameters and be efficient in the use of resources under specified conditions” [3].

Explicación

La eficiencia de rendimiento mide qué tan rápido responde el sistema cuando el usuario hace algo y cuántos recursos consume para lograrlo, ya sea memoria, procesador o ancho de banda. Un sistema que funciona bien en este aspecto no le hace perder el tiempo al usuario esperando respuestas y tampoco sobrecarga el dispositivo

donde corre. Esto se vuelve especialmente crítico en aplicaciones web, donde no todos los usuarios tienen una conexión rápida y el sistema tiene que poder funcionar de forma aceptable incluso en condiciones limitadas.

Ejemplo vinculado al PFC

Cuando un usuario aplica filtros de búsqueda como ubicación, especie, tamaño o edad, el sistema tiene que mostrar los resultados en menos de tres segundos aunque haya miles de mascotas registradas en la base de datos. Y si el usuario está entrando desde el celular con una conexión 4G limitada, las imágenes no pueden cargarse todas de golpe porque eso consume demasiados datos y hace lenta la experiencia. Lo ideal es que carguen de forma progresiva, mostrando primero lo más importante y dejando el resto para después, de manera que el usuario pueda navegar sin sentir que el sistema le está agotando el plan de datos.

2.6.5 Security (ISO/IEC 25010)

Según el modelo de calidad de ISO/IEC 25010, la seguridad (security) es la capacidad de un producto de proteger la información y los datos de modo que las personas o sistemas tengan el grado de acceso adecuado a sus tipos y niveles de autorización [3].

Explicación

La seguridad se encarga de que los datos y las comunicaciones dentro del sistema estén protegidos de accesos no autorizados modificaciones que nadie aprobó o pérdidas de información. En aplicaciones que manejan datos sensibles de los usuarios, como información de contacto o la ubicación de sus mascotas, esto no es algo opcional sino un requisito que tiene que estar bien resuelto desde el diseño. Dentro de esta característica entran cosas como que solo las personas autorizadas puedan ver cierta información, que los datos no puedan ser alterados sin permiso y que las acciones que se realizan dentro del sistema queden registradas para poder rastrearlas si algo sale mal.

Ejemplo vinculado al PFC

En la plataforma solo el dueño que creó el perfil de su mascota puede modificarlo o eliminarlo, el resto de usuarios únicamente puede verlo. Los datos de contacto como el teléfono o el correo no se muestran de entrada, sino que permanecen ocultos hasta que los dos usuarios hayan tenido una conversación dentro de la plataforma, lo que evita que información personal quede expuesta sin que haya un mínimo de interacción previa. Por otro lado, las contraseñas tienen que guardarse encriptadas y las sesiones deben cerrarse solas cuando el usuario lleva un tiempo sin hacer nada, para reducir el riesgo de que alguien acceda a una cuenta que quedó abierta

- [1] Alain. Abran and J. W. . Moore, *Guide to the software engineering body of knowledge : 2004 version : SWEBOK*, 4th ed. IEEE Computer Society Press, 2004.
- [2] ISO/IEC/IEEE, “Systems and software engineering-Life cycle processes-Requirements engineering,” Nov. 2018.
- [3] ISO/IEC, “Systems and software engineering-Systems and software Quality Requirements and Evaluation (SQuaRE)-Product quality model,” Nov. 2023.

